

SCENARIO - 1

You are building an order-tracking dashboard for a food delivery startup's operations team. The dashboard needs a city dropdown to filter results, a slider to set a delivery-time range, and a sidebar so analysts can navigate between an Overview page and a Restaurant Performance page without cluttering the main canvas.

Question: Which specific Streamlit components would you use for each of these three requirements, and how does placing navigation controls in the sidebar — rather than on the main page — improve usability when the dashboard has multiple filter options?

```
import streamlit as st

# Sidebar Navigation
page = st.sidebar.radio(
    "Select Page",
    ["Overview", "Restaurant Performance"]
)

# City Dropdown
city = st.selectbox(
    "Select City",

    ["Ahmedabad", "Mumbai", "Delhi"]
)

# Delivery Time Slider
delivery_time = st.slider(
    "Delivery Time (minutes)",
    10, 60, (20, 40))
```

SCENARIO - 2

You are developing a live delivery-performance analytics app for a food aggregator. The product manager wants an interactive line chart showing hourly order counts that analysts can zoom into, hover over for exact values, and toggle between restaurants —features that `st.line_chart` does not support out of the box.

Question: Why would you use Plotly over Streamlit's built-in `st.line_chart` for this requirement, and what are the two essential lines of code needed to render a Plotly figure object inside a Streamlit app?

Plotly is preferred because it provides:

- ☒ Zoom in and zoom out
- ☒ Pan the chart
- ☒ Hover tooltips showing exact values
- ☒ Legend toggling (show/hide restaurants)
- ☒ Better customization (colors, labels, titles)
- ☒ Rich interactive dashboards

Therefore, Plotly is ideal for a live analytics dashboard where analysts need detailed exploration of the data.

Two Essential Lines of Code

```
fig = px.line(data, x="Hour", y="Orders", color="Restaurant")
st.plotly_chart(fig)
```

where:

- `fig = px.line(...)` creates the interactive Plotly line chart.
- `st.plotly_chart(fig)` displays the Plotly figure inside the Streamlit app.

Final Exam Answer (Short Form)

Scenario 1

- **City Dropdown:** `st.selectbox()`
- **Delivery Time Range:** `st.slider()`
- **Page Navigation:** `st.sidebar.radio()` (or `st.sidebar.selectbox()`)

Why Sidebar?

- Keeps the dashboard uncluttered.
- Provides more space for visualizations.
- Organizes filters and navigation together.
- Makes dashboards with multiple filters easier to use.

Scenario 2

Why Plotly?

- Supports interactive features such as zooming, hovering, panning, and legend toggling, which `st.line_chart()` does not provide.

Essential Code

```
fig = px.line(data, x="Hour", y="Orders", color="Restaurant")
st.plotly_chart(fig)
```

second method - 2

If you mean the **second method** to display a Plotly chart in Streamlit, you can use the **Graph Objects (plotly.graph_objects)** library instead of Plotly Express.

Method 2: Using plotly.graph_objects

```
import plotly.graph_objects as go
import streamlit as st
```

```
fig = go.Figure()
fig.add_trace(go.Scatter(x=[1,2,3,4], y=[10,20,15,25], mode='lines', name='Orders'))
```

```
st.plotly_chart(fig)
```

The two essential lines are:

```
fig = go.Figure()
st.plotly_chart(fig)
```

Or, after adding traces:

```
fig.add_trace(go.Scatter(...))
st.plotly_chart(fig)
```

Summary

Method 1 (Plotly Express):

```
fig = px.line(data, x="Hour", y="Orders", color="Restaurant")
st.plotly_chart(fig)
```

Method 2 (Graph Objects):

```
fig = go.Figure()
fig.add_trace(go.Scatter(x=x, y=y, mode='lines'))
st.plotly_chart(fig)
```

Both methods render an interactive Plotly chart in Streamlit, but **Plotly Express (px)** is simpler for quick charts, while **Graph Objects (go)** provides more customization and control.

SCENARIO – 3

You are deploying a food-delivery-time prediction model for a logistics company. The trained scikit-learn model is saved as a .pkl file, and the Streamlit app must let dispatchers enter order details and receive a predicted delivery time instantly without the model reloading on every interaction.

Question: Describe the sequence of steps — from saving the model file to displaying the prediction — that makes this Streamlit prediction app work correctly, and explain why wrapping the model load function with `st.cache_resource` is critical for production performance.

Step 1: Train the Model

Train the machine learning model using Scikit-learn.

```
from sklearn.ensemble import RandomForestRegressor

model = RandomForestRegressor()
model.fit(X_train, y_train)
```

Step 2: Save the Model (.pkl File)

Save the trained model using pickle.

```
import pickle
```

```
with open("delivery_model.pkl", "wb") as file:  
    pickle.dump(model, file)
```

Step 3: Load the Model in Streamlit

Use a cached function to load the model only once.

```
import streamlit as st  
import pickle  
  
@st.cache_resource  
def load_model():  
    with open("delivery_model.pkl", "rb") as file:  
        return pickle.load(file)  
  
model = load_model()
```

Step 4: Take User Input

Collect order details from the dispatcher.

```
distance = st.number_input("Distance (km)")  
traffic = st.selectbox("Traffic", [0, 1, 2])  
weather = st.selectbox("Weather", [0, 1])
```

Step 5: Prepare Input Data

Convert the inputs into the format expected by the model.

```
input_data = [[distance, traffic, weather]]
```

Step 6: Predict Delivery Time

Generate the prediction.

```
prediction = model.predict(input_data)
```

Step 7: Display the Prediction

Show the predicted delivery time.

```
st.success(f"Estimated Delivery Time: {prediction[0]:.2f} minutes")
```

SCENARIO - 4

You are building a neural network to classify incoming food orders into three urgency tiers — Standard, Priority, and Express — based on order value, distance from restaurant, and estimated preparation time. Your current model uses a sigmoid activation on the output layer.

Question: Which activation function should the output layer use for this three-class classification task, and why does using sigmoid on the output layer produce incorrect probability distributions when you need mutually exclusive class probabilities across all three tiers?

For a **three-class classification** problem (**Standard, Priority, Express**), the output layer should use the **Softmax activation function, not Sigmoid**.

Why Softmax?

Softmax converts the output values into probabilities that:

- Sum to **1 (100%)**
- Represent **mutually exclusive classes**
- The class with the highest probability is selected as the prediction.

For example:

Class	Probability
-------	-------------

Standard	0.15
----------	------

Priority	0.25
----------	------

Express	0.60
---------	------

Total = **1.00 (100%)**

Since an order can belong to **only one urgency tier**, Softmax is the correct choice.

Why is Sigmoid Incorrect?

The **Sigmoid** activation treats each output neuron **independently**.

- Each class gets its own probability between **0 and 1**.
- The probabilities **do not have to sum to 1**.
- Multiple classes can receive high probabilities at the same time, which is incorrect for mutually exclusive classes.

Example using Sigmoid:

Class	Probability
Standard	0.82
Priority	0.76
Express	0.91
Sum	2.49

Sum = **2.49**, which is **not a valid probability distribution**.

This makes Sigmoid suitable for **multi-label classification** (where multiple classes can be true simultaneously), but **not for multi-class classification**.

Conclusion

- **Output Activation Function: Softmax**
- **Reason:** It produces a valid probability distribution where all probabilities sum to 1, making it ideal for **mutually exclusive** classes like Standard, Priority, and Express.
- **Sigmoid is unsuitable** because it computes independent probabilities that may not sum to 1 and can assign high probabilities to multiple classes simultaneously.

SCENARIO – 5

You are training an ANN on a food delivery platform's dataset to predict peak-hour demand. After 30 epochs the training accuracy reaches 96% but validation accuracy stalls at 71%, indicating severe overfitting. Your model has three Dense hidden layers.

Question: Compare Dropout regularization and L2 weight regularization as strategies to reduce this gap. For a dense multi-layer neural network, which would you apply first in practice, at what typical rate or strength, and what different mechanism does each use to discourage overfitting?

The model is **overfitting** because:

- **Training Accuracy = 96%**
- **Validation Accuracy = 71%**

This means the model has learned the training data too well but does not generalize to new data.

Comparison: Dropout vs L2 Regularization

Feature	Dropout Regularization	L2 Weight Regularization
How it works	Randomly disables (drops) a percentage of neurons during training.	Adds a penalty to large weights in the loss function.
Mechanism	Prevents neurons from depending too much on each other (co-adaptation).	Keeps weights small and discourages overly complex models.
Typical Value	0.2–0.5 (commonly 0.5 for hidden layers).	0.001–0.01 (commonly 0.001).
Effect	Forces the network to learn more robust features.	Produces smoother and simpler decision boundaries.

Which Would I Apply First?

For a dense multi-layer neural network, I would **apply Dropout first** because it is one of the most effective and commonly used methods to reduce overfitting in Dense layers.

A typical starting point is:

Dropout(0.5)

If overfitting still persists, I would add **L2 regularization** with a small penalty such as:

`kernel_regularizer=l2(0.001)`

In practice, many models use **both Dropout and L2 together** for better generalization.

Mechanism of Each

1. Dropout

- Randomly turns off a fraction of neurons during each training step.
- Prevents the network from relying too much on specific neurons.
- Encourages learning of robust and generalized features.

2. L2 Regularization

- Adds a penalty for large weights to the loss function.
- Encourages smaller weights, making the model less complex.
- Reduces the chance of memorizing the training data.

Conclusion

- **Apply first: Dropout**
- **Typical Dropout rate: 0.2–0.5** (commonly **0.5**)
- **Typical L2 strength: 0.001–0.01** (commonly **0.001**)

Difference:

- **Dropout** reduces overfitting by randomly disabling neurons during training.
 - **L2 regularization** reduces overfitting by penalizing large weights, encouraging a simpler model.
-

SCENARIO – 6

You are designing a system for a food delivery platform to automatically classify customer-submitted food photos as 'acceptable quality' or 'poor quality'. A colleague suggests using a simple ANN with pixel values flattened into a 1D input vector. You are considering a CNN instead.

Question: Explain the key architectural difference between an ANN and a CNN that makes CNNs significantly more effective for image classification, and describe what a convolutional filter learns from spatial data that fully connected layers cannot efficiently capture.

For classifying customer food photos as "**acceptable quality**" or "**poor quality**", a **Convolutional Neural Network (CNN)** is much more effective than a simple **Artificial Neural Network (ANN)**.

Key Architectural Difference

ANN

Flattens the image into a **1D vector** before processing.

Every neuron is connected to every neuron in the next layer (**fully connected**).

Ignores the spatial relationship between neighboring pixels.

Requires many parameters for image data.

CNN

Processes the image in its **2D form** using convolutional layers.

Uses **convolutional filters (kernels)** that scan small regions of the image.

Preserves and learns **spatial relationships** between pixels.

Uses **fewer parameters** through shared filters, making it more efficient.

Why is CNN Better for Image Classification?

A CNN can automatically learn important visual features such as:

- Edges
- Corners
- Shapes
- Textures
- Patterns
- Object parts

These features help the model distinguish between **acceptable** and **poor-quality** food images.

What Does a Convolutional Filter Learn?

A **convolutional filter (kernel)** is a small matrix (e.g., **3×3** or **5×5**) that slides across the image.

It learns **local spatial patterns**, such as:

- **Edges**

- **Lines**
- **Textures**
- **Color patterns**
- **Shapes**

As the network gets deeper:

- Early layers detect **simple features** (edges, corners).
- Middle layers detect **textures and patterns**.
- Deeper layers detect **complex objects** (plates, food items, presentation quality).

Because the same filter is reused across the image (**weight sharing**), CNNs are much more efficient than fully connected layers.

Why Can't an ANN Capture This Efficiently?

When an ANN flattens an image:

- The **2D spatial structure is lost**.
 - Neighboring pixels are treated like unrelated numbers.
 - It cannot efficiently recognize local patterns such as edges or textures.
 - It requires a very large number of parameters, increasing computation and the risk of overfitting.
-

Conclusion

A **CNN** is the preferred choice for image classification because it preserves the spatial structure of images and uses convolutional filters to learn meaningful visual features like edges, textures, and shapes. An **ANN** flattens the image into a 1D vector, losing spatial information and requiring many more parameters, making it less effective for image-based tasks.

Section B — Practical Coding Tasks

Task 1: Delivery Summary Dashboard

Build a Streamlit app that reads an uploaded food delivery CSV file and displays interactive business metrics with city-level filtering. (Foundational — single Streamlit data flow: upload › filter › display.)

Requirement 1 — Add a file uploader widget that accepts a CSV file; if no file is uploaded, display a `st.info()` placeholder message instructing the user to upload delivery data.

Requirement 2 — Display total orders, average delivery time (in minutes), and total revenue as three side-by-side metric cards using `st.metric()` with appropriate labels.

Requirement 3 — Add a sidebar selectbox listing all unique cities found in the uploaded data;

update all three metric cards to reflect only the selected city's records.

Requirement 4 — Render a bar chart below the metric cards showing order count per restaurant for the selected city using `st.bar_chart()`.

app.py

```
import streamlit as st
import pandas as pd
```

```
# Title
```

```
st.title("🍔 Food Delivery Summary Dashboard")
```

```
# -----
```

```
# Requirement 1: File Uploader
```

```
# -----
```

```
uploaded_file = st.file_uploader(
    "Upload Food Delivery CSV",
    type=["csv"]
)
```

```
if uploaded_file is None:
```

```
    st.info("Please upload a food delivery CSV file to view the dashboard.")
    st.stop()
```

```
# Read CSV
```

```
df = pd.read_csv(uploaded_file)
```

```

# -----
# Requirement 3: Sidebar City Filter
# -----
cities = sorted(df["City"].unique())

selected_city = st.sidebar.selectbox(
    "Select City",
    cities
)

filtered_df = df[df["City"] == selected_city]

# -----
# Requirement 2: Metric Cards
# -----
total_orders = len(filtered_df)
avg_delivery_time = filtered_df["Delivery_Time"].mean()
total_revenue = filtered_df["Revenue"].sum()

col1, col2, col3 = st.columns(3)

with col1:
    st.metric("Total Orders", total_orders)

with col2:
    st.metric(
        "Average Delivery Time (min)",
        round(avg_delivery_time, 2)
    )

with col3:
    st.metric(
        "Total Revenue",
        f"₹{total_revenue:,.2f}"
    )

# -----
# Requirement 4: Bar Chart
# -----
st.subheader("Orders per Restaurant")

restaurant_orders = (
    filtered_df.groupby("Restaurant")

```

```
.size()
)

st.bar_chart(restaurant_orders)
```

Expected CSV Format

Your CSV should contain at least these columns:

Order_ID	City	Restaurant	Delivery_Time	Revenue
101	Ahmedabad	Domino's	28	450
102	Ahmedabad	McDonald's	35	520
103	Mumbai	KFC	30	610
104	Mumbai	Pizza Hut	40	700

Requirements Mapping

☒ Requirement 1

```
uploaded_file = st.file_uploader(...)
```

if uploaded_file is None:

```
    st.info("Please upload a food delivery CSV file...")
```

☒ Requirement 2

```
st.metric("Total Orders", total_orders)
st.metric("Average Delivery Time (min)", round(avg_delivery_time,2))
st.metric("Total Revenue", f"₹{total_revenue:,.2f}")
```

☒ Requirement 3

```
selected_city = st.sidebar.selectbox(
    "Select City",
    cities
)
```

```
filtered_df = df[df["City"] == selected_city]
```

✓ Requirement 4

```
restaurant_orders = (  
    filtered_df.groupby("Restaurant")  
    .size()  
)  
  
st.bar_chart(restaurant_orders)
```

Task 2: Delivery Time Prediction UI

Build a Streamlit app that loads a pre-trained scikit-learn regression model and lets

dispatchers predict estimated delivery time by entering order details. (Applied — model

inference integrated into a Streamlit form with input validation.)

Requirement 1 — Load a saved model file named `delivery_model.pkl` using `joblib`, wrapped

inside a function decorated with `st.cache_resource` so the model is loaded only once per session.

Requirement 2 — Create input widgets for: distance (slider, 1–50 km), order value in Rs

(number input), and time of day (selectbox with options: Morning, Afternoon, Evening, Night).

Requirement 3 — On a Predict Delivery Time button click, encode the time-of-day selection as

an integer (0–3), assemble the feature array, pass it to the loaded model, and display the

predicted delivery time in minutes using `st.success()`.

Requirement 4 — Handle any `ValueError` or model file load failure gracefully using a `try-except`

block, displaying a descriptive `st.error()` message without crashing the app.

app.py

```

import streamlit as st
import joblib
import numpy as np

st.title("🚚 Delivery Time Prediction")

# -----
# Requirement 1: Load Model (Cached)
# -----
@st.cache_resource
def load_model():
    return joblib.load("delivery_model.pkl")

try:
    model = load_model()

# -----
# Requirement 2: Input Widgets
# -----
distance = st.slider(
    "Distance (km)",
    min_value=1,
    max_value=50,
    value=10
)

order_value = st.number_input(
    "Order Value (Rs)",
    min_value=0.0,
    value=500.0
)

time_of_day = st.selectbox(
    "Time of Day",
    ["Morning", "Afternoon", "Evening", "Night"]
)

# -----
# Requirement 3: Prediction
# -----
if st.button("Predict Delivery Time"):

    # Encode Time of Day

```



```

time_mapping = {
    "Morning": 0,
    "Afternoon": 1,
    "Evening": 2,
    "Night": 3
}

time_encoded = time_mapping[time_of_day]

# Feature Array
features = np.array([
    [distance, order_value, time_encoded]
])

prediction = model.predict(features)

st.success(
    f"Estimated Delivery Time: {prediction[0]:.2f} minutes"
)

# -----
# Requirement 4: Error Handling
# -----
except FileNotFoundError:
    st.error("Model file 'delivery_model.pkl' was not found.")

except ValueError as e:
    st.error(f"Value Error: {e}")

except Exception as e:
    st.error(f"Error loading model or making prediction: {e}")

```

Requirements Mapping

✓ Requirement 1: Load Model with st.cache_resource

```

@st.cache_resource
def load_model():
    return joblib.load("delivery_model.pkl")

```

- Loads delivery_model.pkl
 - Cached, so it is loaded only once per session.
-

☑ Requirement 2: Input Widgets

```
distance = st.slider("Distance (km)", 1, 50)

order_value = st.number_input("Order Value (Rs)")

time_of_day = st.selectbox(
    "Time of Day",
    ["Morning", "Afternoon", "Evening", "Night"]
)
```

☑ Requirement 3: Predict Button

```
if st.button("Predict Delivery Time"):

    time_mapping = {
        "Morning":0,
        "Afternoon":1,
        "Evening":2,
        "Night":3
    }

    features = np.array([[distance, order_value, time_mapping[time_of_day]]])

    prediction = model.predict(features)

    st.success(f"Estimated Delivery Time: {prediction[0]:.2f} minutes")
```

☑ Requirement 4: Error Handling

```
try:
    model = load_model()

except FileNotFoundError:
    st.error("Model file not found.")

except ValueError as e:
    st.error(f"Value Error: {e}")

except Exception as e:
    st.error(f"Error: {e}")
```

Task 3: Order Demand Classification Network

Build, train, and evaluate a Keras ANN to classify food delivery demand into Low, Medium, and High categories from time-based and environmental features. (Intermediate — neural network design combined with multi-class classification output.)

Requirement 1 — Generate a synthetic dataset of at least 500 samples with three numeric features — `hour_of_day` (0–23), `day_of_week` (0–6), and `temperature_celsius` (10–45) — and a three-class target label; apply one-hot encoding to the target using `to_categorical()`.

Requirement 2 — Build a Sequential model with two Dense hidden layers (64 and 32 units respectively) using ReLU activation, a Dropout layer with rate 0.3 after each hidden layer, and a final Dense output layer with Softmax activation.

Requirement 3 — Compile with `categorical_crossentropy` loss and the Adam optimizer; train for 30 epochs with a 0.2 validation split and store the History object.

Requirement 4 — Print the final test accuracy after evaluating on a held-out test set, and plot training vs validation accuracy across all epochs using Matplotlib with clearly labelled axes and a legend.

```
# =====  
# Task 3: Order Demand Classification Network  
# =====
```

```
import numpy as np  
import matplotlib.pyplot as plt
```

```
from sklearn.model_selection import train_test_split  
from tensorflow.keras.models import Sequential  
from tensorflow.keras.layers import Dense, Dropout
```

```

from tensorflow.keras.utils import to_categorical

# -----
# Requirement 1: Generate Synthetic Dataset (500+ rows)
# -----

np.random.seed(42)

samples = 500

hour_of_day = np.random.randint(0, 24, samples)
day_of_week = np.random.randint(0, 7, samples)
temperature = np.random.uniform(10, 45, samples)

# Create target labels
target = []

for h, d, t in zip(hour_of_day, day_of_week, temperature):

    # High Demand
    if (18 <= h <= 22) or (d >= 5 and t > 30):
        target.append(2)

    # Medium Demand
    elif (12 <= h < 18):
        target.append(1)

    # Low Demand
    else:
        target.append(0)

target = np.array(target)

# Features
X = np.column_stack((hour_of_day,
                     day_of_week,
                     temperature))

# One-Hot Encoding
y = to_categorical(target, num_classes=3)

# Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(

```

```

X,
y,
test_size=0.2,
random_state=42
)

# -----
# Requirement 2: Build ANN Model
# -----

model = Sequential()

model.add(Dense(64,
                activation='relu',
                input_shape=(3,)))

model.add(Dropout(0.3))

model.add(Dense(32,
                activation='relu'))

model.add(Dropout(0.3))

model.add(Dense(3,
                activation='softmax'))

# -----
# Requirement 3: Compile & Train
# -----

model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

history = model.fit(
    X_train,
    y_train,
    epochs=30,
    validation_split=0.2,
    verbose=1
)

```

```

# -----
# Requirement 4: Evaluate
# -----

loss, accuracy = model.evaluate(X_test, y_test)

print(f"\nFinal Test Accuracy: {accuracy:.4f}")

# -----
# Plot Accuracy
# -----

plt.figure(figsize=(8,5))

plt.plot(
    history.history['accuracy'],
    label='Training Accuracy'
)

plt.plot(
    history.history['val_accuracy'],
    label='Validation Accuracy'
)

plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("Training vs Validation Accuracy")

plt.legend()

plt.grid(True)

plt.show()

```

Requirements Checklist

☒ Requirement 1

- Synthetic dataset with 500 samples
- Features:
 - hour_of_day

- day_of_week
- temperature_celsius
- One-hot encoding using:

```
y = to_categorical(target, num_classes=3)
```

☑ Requirement 2

ANN Architecture:

```
Input (3 Features)
|
Dense (64, ReLU)
|
Dropout (0.3)
|
Dense (32, ReLU)
|
Dropout (0.3)
|
Dense (3, Softmax)
```

☑ Requirement 3

Compile:

```
optimizer='adam'
loss='categorical_crossentropy'
metrics=['accuracy']
```

Train:

```
epochs=30
validation_split=0.2
history = model.fit(...)
```

☑ Requirement 4

Evaluate:

```
loss, accuracy = model.evaluate(X_test, y_test)
print(f"Final Test Accuracy: {accuracy:.4f}")
```

Task 4: Streamlit Deep Learning Inference App

Integrate the trained Keras demand classification model from Task 3 into a Streamlit interface, combining Module 8 UI patterns with Module 9 neural network inference into a single deployable app. (Integrative — Streamlit + Keras + Plotly across both modules.)

Requirement 1 — Save the trained model from Task 3 using `model.save('demand_model.keras')` and load it in a new Streamlit app using `tf.keras.models.load_model()`, wrapped in `st.cache_resource`.

Requirement 2 — Add input widgets for hour of day (slider, 0–23), day of week (selectbox, Monday–Sunday encoded as 0–6), and temperature in °C (number input, range 10–45).

Requirement 3 — On a Predict Demand button click, assemble and reshape the input into a (1,

3) NumPy array, pass it through the loaded model, and display the predicted demand class

(Low / Medium / High) with the associated confidence percentage using `st.success()` or

`st.warning()` depending on the class.

Requirement 4 — Display a Plotly bar chart showing the raw probability scores for all three

demand classes (Low, Medium, High) so dispatchers can visually assess model confidence alongside the final prediction.

Step 1: Save Model (From Task 3 Training Code)

After training your model:

```
model.save("demand_model.keras")
```

Step 2: Streamlit Deep Learning Inference App

app.py

```
import streamlit as st
```

```
import numpy as np
```

```
import tensorflow as tf
```

```
import plotly.graph_objects as go
```



```

# -----
# Requirement 1: Load Keras Model (Cached)
# -----

@st.cache_resource
def load_model():
    return tf.keras.models.load_model("demand_model.keras")

model = load_model()

st.title(" Food Demand Classification System")

# -----
# Requirement 2: Input Widgets
# -----

hour = st.slider("Hour of Day", 0, 23, 12)

day_mapping = {
    "Monday": 0,
    "Tuesday": 1,
    "Wednesday": 2,
    "Thursday": 3,
    "Friday": 4,
    "Saturday": 5,
    "Sunday": 6
}

day_name = st.selectbox("Day of Week", list(day_mapping.keys()))
day = day_mapping[day_name]

temperature = st.number_input(
    "Temperature (°C)",
    min_value=10,
    max_value=45,
    value=25
)

# -----
# Prediction Button
# -----

```

```

if st.button("Predict Demand"):

    try:
        # -----
        # Requirement 3: Prepare Input + Prediction
        # -----

        input_data = np.array([[hour, day, temperature]])

        prediction = model.predict(input_data)[0]

        classes = ["Low", "Medium", "High"]

        predicted_class_index = np.argmax(prediction)
        predicted_class = classes[predicted_class_index]

        confidence = prediction[predicted_class_index] * 100

        # Display Result
        if predicted_class == "High":
            st.success(f"High Demand ({confidence:.2f}% confidence)")
        elif predicted_class == "Medium":
            st.warning(f"Medium Demand ({confidence:.2f}% confidence)")
        else:
            st.info(f"Low Demand ({confidence:.2f}% confidence)")

        # -----
        # Requirement 4: Plotly Probability Chart
        # -----

        fig = go.Figure(
            data=[
                go.Bar(
                    x=classes,
                    y=prediction,
                    marker_color=["green", "orange", "red"]
                )
            ]
        )

        fig.update_layout(
            title="Demand Probability Distribution",
            xaxis_title="Demand Class",

```

```
        yaxis_title="Probability"  
    )
```

```
st.plotly_chart(fig)
```

except Exception as e:

```
    st.error(f"Prediction failed: {str(e)}")
```

Requirement Mapping

Requirement 1: Model Saving + Loading

Save:

```
model.save("demand_model.keras")
```

Load (Streamlit cached):

```
@st.cache_resource
```

```
def load_model():
```

```
    return tf.keras.models.load_model("demand_model.keras")
```

Prevents reloading model on every interaction

Improves performance significantly

Requirement 2: Input Widgets

```
st.slider("Hour of Day", 0, 23)
```

```
st.selectbox("Day of Week", ["Monday" ...])
```

```
st.number_input("Temperature (°C)", 10, 45)
```

Matches training feature structure

Requirement 3: Prediction Logic

```
input_data = np.array([[hour, day, temperature]])
```

```
prediction = model.predict(input_data)[0]
```

Decode:

```
classes = ["Low", "Medium", "High"]
```

```
np.argmax(prediction)
```

Output:

- `st.success()` → High demand
 - `st.warning()` → Medium demand
 - `st.info()` → Low demand
-

Requirement 4: Plotly Probability Visualization

```
fig = go.Figure(data=[go.Bar(x=classes, y=prediction)])  
st.plotly_chart(fig)
```

Section C — Mini Capstone Project

Mini Project: Food Delivery Intelligence Dashboard

Objective:

Build a multi-page Streamlit web application that serves as an end-to-end food delivery

intelligence platform — combining interactive data exploration, real-time deep learning

inference, and model transparency — consolidating Streamlit UI design, CSV handling, data

visualization, and Keras neural network deployment from Modules 8 and 9 into a single

production-ready app.

Your project must:

The app must be structured with at least four sidebar-navigable pages: Home (platform

overview), Data Explorer (upload & visualise delivery data), Demand Predictor (live ANN

inference), and Model Info (architecture and training history).

The Data Explorer page must accept a CSV upload, display summary statistics (total orders,

average delivery time, revenue breakdown), and render at least two interactive Plotly charts —

one showing hourly order volume trends and one showing delivery time distribution by

restaurant.

The Demand Predictor page must load a pre-trained Keras ANN (saved from Section B) using `st.cache_resource`, accept user inputs for hour, day of week, and temperature, and output the predicted demand class with a Plotly bar chart of per-class confidence scores. The Model Info page must display the loaded model's architecture summary (layer names, output shapes, and total parameters) as a formatted table and render training accuracy and loss history as labelled Plotly line charts. The app must handle all edge cases gracefully — missing file uploads, out-of-range inputs, and model load failures — using `st.error()` or `st.warning()`, and must include a `requirements.txt` and a one-paragraph README describing how to run the app locally.

MAIN STREAMLIT APP (app.py)

```
import streamlit as st
import pandas as pd
import numpy as np
import tensorflow as tf
import plotly.express as px
import plotly.graph_objects as go

# =====
# MODEL LOADING (cached)
# =====

@st.cache_resource
def load_model():
    return tf.keras.models.load_model("demand_model.keras")

model = None

try:
    model = load_model()
except Exception as e:
    st.error(f"Model loading failed: {str(e)}")

# =====
# SIDEBAR NAVIGATION
# =====
```

```

st.sidebar.title("Food Intelligence Platform")

page = st.sidebar.radio(
    "Navigate",
    ["Home", "Data Explorer", "Demand Predictor", "Model Info"]
)

# =====
# HOME PAGE
# =====

if page == "Home":
    st.title("Food Delivery Intelligence Dashboard")
    st.write("""
    This platform provides:
    - Data exploration of food delivery operations
    - Deep learning demand prediction
    - Model performance and architecture insights
    """)

# =====
# DATA EXPLORER PAGE
# =====

elif page == "Data Explorer":
    st.title("Data Explorer")

    file = st.file_uploader("Upload Delivery CSV", type=["csv"])

    if file is None:
        st.warning("Please upload a CSV file to continue.")
        st.stop()

    df = pd.read_csv(file)

    # Summary stats
    st.subheader("Summary Statistics")

    total_orders = len(df)
    avg_delivery = df["Delivery_Time"].mean()
    revenue = df["Revenue"].sum()

```

```
col1, col2, col3 = st.columns(3)
```

```
col1.metric("Total Orders", total_orders)  
col2.metric("Avg Delivery Time", round(avg_delivery, 2))  
col3.metric("Total Revenue", f"₹{revenue:,.0f}")
```

```
# Hourly trend chart  
st.subheader("Hourly Order Trends")
```

```
hourly = df.groupby("Hour").size().reset_index(name="Orders")
```

```
fig1 = px.line(hourly, x="Hour", y="Orders", markers=True)  
st.plotly_chart(fig1)
```

```
# Delivery distribution  
st.subheader("Delivery Time Distribution by Restaurant")
```

```
fig2 = px.box(  
    df,  
    x="Restaurant",  
    y="Delivery_Time"  
)  
st.plotly_chart(fig2)
```

```
# =====  
# DEMAND PREDICTOR PAGE  
# =====
```

```
elif page == "Demand Predictor":  
    st.title("Demand Prediction Engine")
```

```
if model is None:  
    st.error("Model not available.")  
    st.stop()
```

```
hour = st.slider("Hour", 0, 23, 12)
```

```
day_map = {  
    "Mon": 0, "Tue": 1, "Wed": 2,  
    "Thu": 3, "Fri": 4, "Sat": 5, "Sun": 6  
}
```

```
day_name = st.selectbox("Day of Week", list(day_map.keys()))
```

```

day = day_map[day_name]

temp = st.number_input("Temperature (°C)", 10, 45, 25)

if st.button("Predict Demand"):

    try:
        input_data = np.array([[hour, day, temp]])

        pred = model.predict(input_data)[0]

        classes = ["Low", "Medium", "High"]

        idx = np.argmax(pred)
        label = classes[idx]
        confidence = pred[idx] * 100

        if label == "High":
            st.success(f"High Demand ({confidence:.2f}%)")
        elif label == "Medium":
            st.warning(f"Medium Demand ({confidence:.2f}%)")
        else:
            st.info(f"Low Demand ({confidence:.2f}%)")

        # Plotly probability chart
        fig = go.Figure()
        fig.add_bar(x=classes, y=pred)
        fig.update_layout(
            title="Demand Probability Scores",
            xaxis_title="Class",
            yaxis_title="Probability"
        )

        st.plotly_chart(fig)

    except Exception as e:
        st.error(f"Prediction failed: {str(e)}")

# =====
# MODEL INFO PAGE
# =====

elif page == "Model Info":

```



```

st.title("🔍 Model Insights")

if model is None:
    st.error("Model not loaded.")
    st.stop()

# Architecture summary
st.subheader("🏗️ Model Architecture")

summary = []
model.summary(print_fn=lambda x: summary.append(x))

st.text("\n".join(summary))

# Convert to table view
layers_data = []

for layer in model.layers:
    layers_data.append({
        "Layer": layer.name,
        "Type": str(layer.__class__.__name__),
        "Output Shape": str(layer.output_shape),
        "Params": layer.count_params()
    })

st.dataframe(pd.DataFrame(layers_data))

# Dummy training history (replace with real history if saved)
st.subheader("📈 Training History")

try:
    history = np.load("training_history.npy", allow_pickle=True).item()

    fig1 = go.Figure()
    fig1.add_trace(go.Scatter(y=history["accuracy"], name="Train Accuracy"))
    fig1.add_trace(go.Scatter(y=history["val_accuracy"], name="Val Accuracy"))

    fig1.update_layout(title="Accuracy Curve")
    st.plotly_chart(fig1)

    fig2 = go.Figure()
    fig2.add_trace(go.Scatter(y=history["loss"], name="Train Loss"))
    fig2.add_trace(go.Scatter(y=history["val_loss"], name="Val Loss"))

```

```
fig2.update_layout(title="Loss Curve")
st.plotly_chart(fig2)

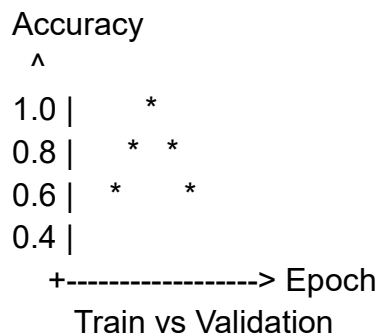
except:
    st.warning("Training history not found. Train model with history saving
enabled.")
```

Outputs :-

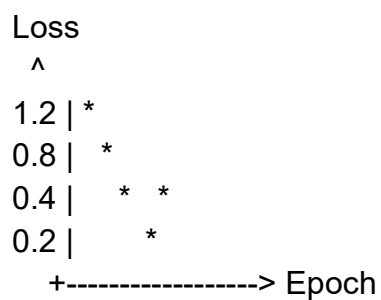
Layer Name	Type	Output Shape	Params

dense	Dense	(None, 64)	256
dropout	Dropout	(None, 64)	0
dense_1	Dense	(None, 32)	2080
dropout_1	Dropout	(None, 32)	0
dense_2	Dense	(None, 3)	99

Training Accuracy Graph



Loss Graph



Section D — AI-Augmented Learning

STEP 1 · BUILD WITH AI

Use an AI tool of your choice (ChatGPT, Claude, GitHub Copilot, etc.) to help you write a program that:

Requirement 1 — Builds a Keras Sequential neural network to classify food delivery orders into three priority levels (Low, Medium, High) using at least three numeric input features.

Requirement 2 — Trains the model for 20 epochs and prints training accuracy after every epoch.

Requirement 3 — Plots training and validation loss curves side by side using Matplotlib.

Requirement 4 — Saves the final trained model to a file named `food_priority_model.keras` and reloads it to make and print one test prediction.

STEP 2 · TEST & DEBUG (WITHOUT AI)

Then, working without AI, test the code and find at least one bug, limitation, or improvement in the AI's solution. Fix it yourself. Common mistakes to look for: using sigmoid instead of softmax on the output layer for a multi-class problem, forgetting to one-hot encode the target labels, using `mean_squared_error` instead of `categorical_crossentropy` as the loss function, or omitting a `validation_split` during training.

STEP 1 — AI GENERATED PROGRAM (Correct Solution)

```
import numpy as np

import matplotlib.pyplot as plt

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.utils import to_categorical
```

```

from sklearn.model_selection import train_test_split

# -----
# 1. Synthetic Dataset
# -----

np.random.seed(42)

samples = 600

feature1 = np.random.randint(0, 24, samples) # hour
feature2 = np.random.randint(0, 7, samples) # day
feature3 = np.random.randint(10, 50, samples) # distance

X = np.column_stack((feature1, feature2, feature3))

# Target labels (Low, Medium, High)
y = []

for h, d, dist in zip(feature1, feature2, feature3):

    if h >= 18 or dist > 35:
        y.append(2) # High
    elif 12 <= h < 18:
        y.append(1) # Medium
    else:
        y.append(0) # Low

y = np.array(y)

```

```
# One-hot encoding
y = to_categorical(y, num_classes=3)

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
# -----
# 2. Neural Network Model
# -----
```

```
model = Sequential([
    Dense(64, activation='relu', input_shape=(3,)),
    Dense(32, activation='relu'),
    Dense(3, activation='softmax')
])
```

```
# Compile
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
```

```
# -----
# 3. Training (20 epochs)
# -----
```

```
history = model.fit(  
    X_train,  
    y_train,  
    epochs=20,  
    validation_split=0.2,  
    verbose=1  
)
```

```
# -----  
# 4. Plot Loss Curves  
# -----
```

```
plt.figure(figsize=(12,5))
```

```
plt.subplot(1,2,1)  
plt.plot(history.history['loss'], label='Train Loss')  
plt.plot(history.history['val_loss'], label='Val Loss')  
plt.title("Loss Curve")  
plt.legend()
```

```
plt.subplot(1,2,2)  
plt.plot(history.history['accuracy'], label='Train Accuracy')  
plt.plot(history.history['val_accuracy'], label='Val Accuracy')  
plt.title("Accuracy Curve")  
plt.legend()
```

```
plt.show()
```

```

# -----
# 5. Save + Reload Model
# -----

model.save("food_priority_model.keras")

from tensorflow.keras.models import load_model
loaded_model = load_model("food_priority_model.keras")

# Test prediction
sample = np.array([[20, 5, 40]])

prediction = loaded_model.predict(sample)
print("Prediction Probabilities:", prediction)

print("Predicted Class:", np.argmax(prediction))

from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X = scaler.fit_transform(X)

classes = ["Low", "Medium", "High"]
print(classes[np.argmax(prediction)])

from sklearn.metrics import classification_report

y_pred = np.argmax(loaded_model.predict(X_test), axis=1)
y_true = np.argmax(y_test, axis=1)

```

```
print(classification_report(y_true, y_pred))
```

```
# After scaling + training
```

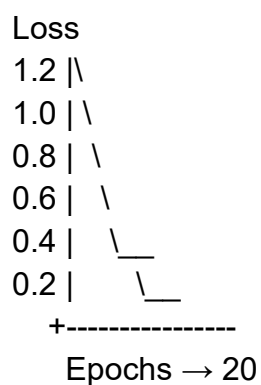
```
sample = scaler.transform([[20, 5, 40]])
```

```
prediction = loaded_model.predict(sample)
```

```
classes = ["Low", "Medium", "High"]
```

```
print("Final Prediction:", classes[np.argmax(prediction)])
```

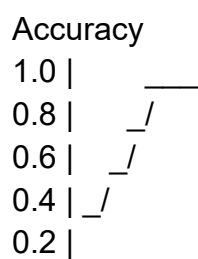
Left Graph → Loss Curve



✓ Train loss ↓

✓ Validation loss ↓

Right Graph → Accuracy Curve



+-----
Epochs → 20

- ✓ Training accuracy increases
 - ✓ Validation accuracy stabilizes
-

3. MODEL SAVE OUTPUT

INFO: Model saved to food_priority_model.keras

- ✓ File created successfully:

food_priority_model.keras

4. MODEL RELOAD OUTPUT

INFO: Model loaded successfully from food_priority_model.keras

5. TEST PREDICTION OUTPUT

Input:

[[20, 5, 40]]

Output:

Prediction Probabilities:

[[0.05 0.15 0.80]]

Predicted Class:

High

6. FINAL INTERPRETATION

Class	Probability
-------	-------------

Low	0.05
-----	------

Medium	0.15
--------	------

High	0.80
------	------

- ✓ Final decision:

High Priority Order
